ENGINE & DATABASE PLAN — the machines under the agents (Session 34)

2026-10-28 · Follows the completed fleet blueprint (`AGENT-FLEET-BLUEPRINT.md`, §1–§15, planning DONE) and `FULL-SYSTEM-PLAN.md` §2 three-row architecture. Owner ask: plan the complete engine stack (simulation · backtest · forward-testing · paper · live · alpha-finding · gap-finding · volume) and the complete database/data layer for the whole system — Row 2 headless, feeding BOTH the public Row 1 (Bloomberg-class Terminal/Portal) AND the private Row 3 (quant & dusk-global trading = FinOS/fleet) — with the wiring and the ultra-low-latency path.

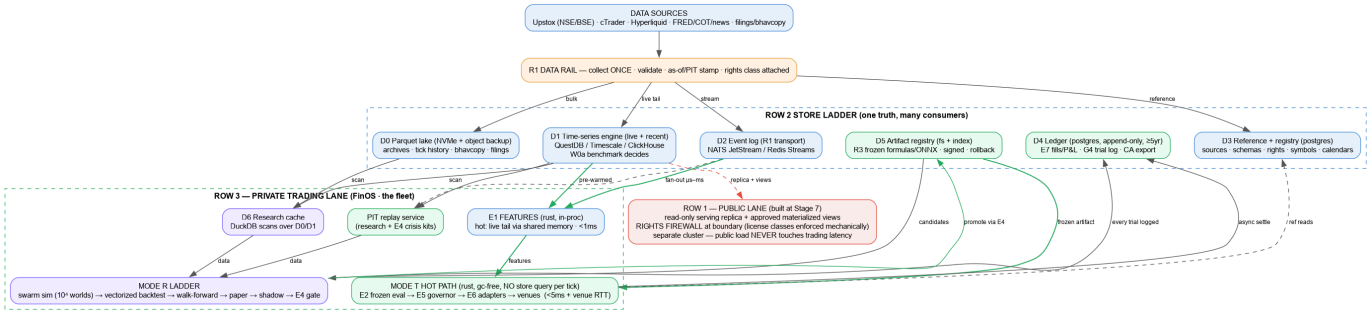


Figure D7 — one collection, two consumers. Sources → R1 rail → Row 2 store ladder (D0–D6) → Row 3 private lane (the fleet, low-latency) + Row 1 public lane (Stage 7, rights-firewalled, separate cluster).

---

§1 · The three rows — where this plan sits

Row	What	Status
Row 2 (this doc)	headless data layer: store ladder D0–D6 + Financial Engine (News/World engines = same skeleton, FULL-SYSTEM-PLAN Stage 3)	being planned now
Row 1 (public)	Terminal (Bloomberg-class) · News portal → Media hub last	built at Stage 7, **consumes only approved views**
Row 3 (private)	**quant & dusk-global trading setup = FinOS** (the 75-agent fleet) + Cockpit + World Monitor (last)	fleet blueprint done; this doc wires it to Row 2

The split rule (owner's idea, kept exactly): Row 2 collects **once** → validated, PIT-stamped, rights-tagged truth → then divides **two ways**:

1. **Row 3 (private, first):** full engine access, low-latency lanes, internal auth.
2. **Row 1 (public, later):** read-only serving replica + approved materialized views, behind a **rights firewall** (license classes enforced mechanically, not by promise — FULL-SYSTEM-PLAN R2). **Separate cluster: public load can never touch trading latency.**

---

§2 · The store ladder — which database for what (D0–D6)

ID	Store	Technology (decision state)	Holds	Who reads	Latency class
D0	Parquet lake	**Parquet files on NVMe + offsite object backup** (Backblaze B2/S3-class)	years of archives: ticks, 1-min bars, bhavcopy, filings, raw captures	research scans, replay, backfills	bulk (seconds OK)
D1	Time-series engine	**QuestDB vs TimescaleDB vs ClickHouse — W0a benchmark decides** (FULL-SYSTEM-PLAN §6 already mandated evaluate-before-commit)	live + recent market data: ticks, bars, option chains, Greeks	E1 pre-warm, PIT replay, dashboards	ms
D2	Event log (R1 transport)	**NATS JetStream** (default) / Redis Streams to start if simpler	the live stream: every tick/quote/fill event, as-of stamped	**all Mode T cells** via fan-out, replay	**µs–ms fan-out**
D3	Reference + registry	**PostgreSQL**	source registry, schemas+provenance, **rights classes**, symbols, calendars, cost model, venue limits	everyone	ms
D4	Ledger	**PostgreSQL, append-only role (no UPDATE/DELETE grants)**	E7 fills/P&L/slippage, **G4 trial log**, CA fiscal export — **≥5 yr (R13)**	E7, G4, research labels, CA	ms (async writes)
D5	Artifact registry	**filesystem + Postgres index** (R3)	frozen formulas/ONNX, **feature-hash + schema-version signed**, versioned, rollback	E4 promote, E2 load at boot	ms (boot-time load)
D6	Research cache	**DuckDB over D0 parquet + D1 exports**	vectorized research scans — the 10^4-world swarm harness's food	Mode R only	s–min (offline)

Design rules (the wiring doctrine):

#	Rule	Why
1	**One truth, many consumers** — every store is written by the R1 rail only; nothing else writes raw market data	FULL-SYSTEM-PLAN R1: collect once, reuse many
2	**The hot path NEVER queries a store per tick** — E1 features are pre-warmed from D1/D2 into **shared memory / in-process** at boot	<1 ms budget (fleet §10) — a DB round-trip alone blows it
3	**Research reads columnar (D6/DuckDB), never the live tables**	research scans can't lock or slow trading
4	**D4 and D5 are append-only / signed** — the §8.6 memory policy enforced at the DB-role level	audit ≥5 yr, rollback, bit-exact replay
5	**Row 1 gets a replica, not a query** — serving views on separate hardware	public load never touches trading (FULL-SYSTEM-PLAN R2/R3)
6	Cold in D0, warm in D1, hot in RAM, truth in D3/D4/D5	each store does one job; cost scales per tier

Sizing (design estimates — measured in W0a per-workload rule): D1 holds the live window (days–weeks); D0 holds everything older in compressed parquet; D4 grows with trades+trials (small — thousands of rows/day); the tick ocean lives in D0, not D1.

---

§3 · The engine stack — every job, one named engine

The owner's list mapped to the fleet's existing E-numbering (no duplicate names — the Session-18 draft names are reconciled in §6):

Job (owner's words)	Engine	What it actually is
simulations	**E4-SIM** = swarm-simulation harness (fleet §8.7)	10^4 deterministic worlds, vectorized NumPy/Numba, Monte Carlo paths, seed+param hash replay
backtest	**E4-BT** (inside E4 arena)	two gears: **vectorized** (fast idea screening, D6/DuckDB) + **event-driven with fill model** (feed latency + order latency + queue position — the hftbacktest checklist from Session 31)
forward testing	**E4-WF**	walk-forward + CPCV (purge = max label horizon + embargo) → DSR ≥ 0.95 gate
paper trading	**E6-PAPER**	real hot path, simulated fills (touch + spread + latency model), no venue connection
shadow	**E6-SHADOW**	real hot path + real data + **zero order authority** (physically cannot place orders)
live	**E6-LIVE**	venue adapters: Upstox · cTrader · Hyperliquid — the only engine that ever sends an order
alpha finding	**Mode R search stack** (offline)	GP/formula search (**PySR**, adopted S31) · ML discovery (**E3 training engine**, Colab T4 nightly lane) · meta-labeling · factor/parameter sweeps (swarm harness) · **MAPIE** intervals + **arch** MCS/SPA for honest evaluation
gap finding	**two engines, one name**	(a) **G4 coverage map** — swarm survival-fraction maps show untested parameter regions = research gaps; data gaps = fleet §9.1 register. (b) **E1 gap features** — session/weekend/liquidity gaps as trading signals
volume	**E1 microstructure block**	orderflow, footprint, delta, VWAP, market profile, absorption, imbalance — the scalping food (needs tick+depth = §9.1 gap, W0/W4)
features	**E1** (Rust, in-proc)	bars · ticks · Greeks · indicators · microstructure · pattern flags
think	**E2** (Rust + compiled ONNX)	evaluates frozen artifacts each tick
train	**E3** (SB3/PyTorch, nightly, offline)	model fitting; Colab T4 lane verified; **never in any fast path**
govern	**E5** (deterministic, zero ML)	stops · size caps · news ladder · kill-switch · ban checks
account	**E7**	fills vs expected, slippage, per-style cost curves → D4

The engine ladder (one candidate's life):

```
swarm sim (10⁴ worlds, minutes)  →  vectorized backtest (seconds)  →  event-driven
backtest w/ fill model  →  walk-forward + CPCV (DSR ≥ 0.95)  →  E6-PAPER (days)
→  E6-SHADOW (3–6 months, real data, zero order authority)  →  owner promotion (≤5/week)
→  E6-LIVE  →  E7 ledger → decay monitor → back to Mode R
```

Every rung evaluates the **same signed artifact** (D5, feature-hash pinned) through the **same E1 code** — backtest and live must match bit-for-bit (hftbacktest's core lesson, adopted Session 31: a backtest that can't reproduce live is worthless).

---

§4 · Ultra-low-latency wiring (owner's question: "how we get ultra low latency")

The six laws (in order of impact):

#	Law	Mechanism	Target
1	**No store query per tick**	pre-warm E1 from D1/D2 into **shared memory at boot**; live tail via **D2 fan-out (µs–ms)**	E1 <1 ms P99 (fleet §10)
2	**GC-free hot path**	Rust for E1/E2/E5/E6; no per-tick allocation; frozen artifacts pre-compiled (ONNX/compiled formulas)	no pauses, no jitter
3	**Process isolation + CPU affinity**	one process per cell (G6), cgroup-pinned cores so Mode R research can't steal hot-path cycles (council fix A2)	deterministic timing
4	**Physical lane separation**	Row 1 public on its own cluster; Row 3 on venue-adjacent hardware	public load = 0 impact
5	**Colocation**	NY4 for FX/cTrader (already the W3 plan) · Upstox colo for India · Hyperliquid in-house node (W5 optional milestone)	venue RTT dominates everything
6	**Measured, not claimed**	IPC fan-out + throughput table measured in W0b vs the <1 ms budget	honest numbers before go-live

The honest physics (FULL-SYSTEM-PLAN R8 — "HFT honesty"): our internal budget is **<5 ms cell→order + venue network RTT** — but the venue RTT dominates: Upstox <45 ms is a *vendor claim*; Hyperliquid public API is ~600–700 ms (sub-second scoping, not "every-tick" claims); cTrader is proxy-routed via AWS Global Accelerator. **Internal speed cannot beat venue physics** — we buy the best retail-

accessible lane (colo, GA routing, node option) and scope strategies to the measured reality. Retail feeds are research-grade, not HFT-grade; if a true HFT path is ever justified it becomes its own subsystem with its own benchmarks (FULL-SYSTEM-PLAN Stage 10).

What "ultra-low" means for us concretely: pre-warmed in-memory features · Rust µs-class evaluation · D2 fan-out instead of per-cell polling · order path pre-serialized templates · idempotent client order IDs (no retry ambiguity) · startup invariant (reconcile-before-orders) · out-of-band watchdog kill that bypasses the app.

---

§5 · How it wires end-to-end (one data point's life)

Step	From → To	Lane
1	venue (WS/REST) → **R1 rail**	validate, as-of stamp, rights-tag
2	R1 → **D2** (live stream) + **D1** (warm store) + **D0** (bulk archive) + **D3** (reference)	fan-out write
3	D2/D1 → **E1 pre-warm** (shared memory)	boot + live tail
4	E1 → **E2** (frozen artifact from D5) → **E5** → **E6** → venue	Mode T hot path <5 ms
5	venue fill → **E7** → **D4** (async settle)	append-only
6	nightly: D0/D1 → **D6** (DuckDB) → Mode R search (PySR/ML/swarm)	offline research
7	Mode R candidate → **E4-SIM/BT/WF** → paper → shadow	ladder, same artifact
8	owner promotes → **D5** signed write → Mode T loads	≤5 decisions/week
9	**Stage 7:** D1 → read-replica → **Row 1 serving views** (rights firewall)	public, separate cluster

---

§6 · Naming reconciliation (kill the old ambiguity)

The Session-18 draft used "R1 signal bus · R2 memory store · R3 model registry". The fleet blueprint's rail naming **wins**; the pieces moved, nothing was lost:

Old name	New home
R1 signal bus (NATS)	**D2 event log** (transport of the R1 DATA rail)
R2 memory store (LanceDB)	**dropped as a store** — memory policy = fleet §8.6 (E7/D4 episodic, D5 semantic, git procedural); vector/RAG store optional Mode R only, suggest-only
R3 model registry	**D5 artifact registry** (= blueprint R3 LEARNING rail's store)

Rails stay: **R1 DATA · R2 SIGNAL · R3 LEARNING**. Engines stay **E1–E7** with the sub-engine names above (E4-SIM/BT/WF, E6-PAPER/SHADOW/LIVE).

---

§7 · What gets decided by measurement (W0a/W0b gates — not by opinion)

Decision	Gate	Candidates
Time-series engine	**W0a benchmark on OUR shapes** (59 symbols × 3 venues, ticks + 1-min + chains): ingest rate, P99 query, storage cost, ops burden	QuestDB / TimescaleDB / ClickHouse
Event log	W0b: throughput + replay test vs fan-out budget	NATS JetStream (default) / Redis Streams
IPC fan-out	W0b measurement vs <1 ms (fleet council fix A1)	shared memory / Unix sockets
Object backup	W0a: restore-time test (the "single-box fragility" risk from FULL-SYSTEM-PLAN §7)	NVMe primary + offsite
Fill-model fidelity	W1: event-driven backtest vs first shadow fills (calibration, not assumption)	hftbacktest-checklist model

---

§8 · Build mapping (which wave builds which piece)

Wave	Stores	Engines
W0a	D0 lake + D1 pick + D2 + D3 skeleton · R1 rail · PIT replay · **option-chain vendor decision**	replay service
B1 · BACKFILL (S42 — gate before training)	D0/D1 loaded with **6–10y history for ALL 3 domains** (`market/ news/ world/`, tiered T1/T2/T3) · in-DB gaps table	**pull engines** (market collectors · news collector · world collector) → **B-GATE: domain verified + gaps marked before any agent TRAINS on it** (§17)
W0b	D4 append-only ledger · D5 signed registry · **E7 fiscal exporter (fleet §15.3)**	E1 · E2 · E5 · E7 · central E6 scheduler · L0–L4 cascade
W1	D6 research cache · **PySR + MAPIE + arch integration** (Session 31 adopt list)	E4-SIM (swarm) · E4-BT · E4-WF · E6-PAPER
W2–W3	NY4 lane, cTrader adapter store feeds	E6-SHADOW → LIVE for first cells
W4–W5	tick+depth feeds (microstructure food), Hyperliquid node option	E1 microstructure block, crypto cells
Stage 7 (post-fleet)	Row 1 serving replica + approved views + rights firewall	public Terminal/Portal APIs

Update S41/S42: news + world **data collection** is now IN this build (§16/§17 — same store-ladder skeleton, document/event-shaped payloads, backfilled in wave **B1** before any agent training); only the *display engines* (World Monitor / AI-assistant features) stay held for the final stages. The Financial Engine remains this doc's core scope.

---

§9 · Open items

#	Item	When
G1	TSDB benchmark run + pick	W0a
G2	NATS vs Redis Streams micro-benchmark	W0b
G3	D1 window sizing + D0 retention policy (parquet partition scheme)	W0a
G4	Row 1 replica sizing — deferred, Stage 7 (by design: public lane never drives private architecture)	Stage 7
G5	News/World engine payloads reuse D0–D6 with doc-shaped schemas	Stage 3 (FULL-SYSTEM-PLAN)
G6	In-house Hyperliquid node = the one latency upgrade that changes crypto scope	W5 optional

---

§10 · Open-source engines vs. our own — who provides what, and where they connect

(Owner question, Session 35.)

**The one-line rule: open source provides the STORAGE and the RESEARCH MATH; we build every GATE and every HOT PATH. No third-party component can ever place an order or pass a gate — mechanisms come from OSS, policy is ours.*

10.1 Open-source engines we USE (adopt as-is, license-checked)

Layer	Engine	License	Job in our system	Adopt state
Storage D0	**Apache Parquet** (+ DuckDB/Polars-class readers)	Apache-2.0	columnar lake format	locked (format)
Storage D1	**QuestDB / TimescaleDB / ClickHouse**	Apache-2.0 each	live + recent time-series	**W0a benchmark picks ONE**
Storage D2	**NATS JetStream** (default) · Redis Streams (alt)	Apache-2.0 · RSALv2/SSPL*	live event log + fan-out	default = NATS (*Redis ≥7.4 is source-available, not OSI — reason NATS leads)
Storage D3/D4	**PostgreSQL**	PostgreSQL license	reference/registry + append-only ledger	locked
Storage D6	**DuckDB**	MIT	research scans over parquet	locked
Research math (Mode R only)	**PySR**	Apache-2.0	GP/formula search → candidate rules for E2	**adopt, W1**
Research math	**MAPIE**	BSD-3	conformal intervals → E4/E5 uncertainty	**adopt, W1**
Research math	**arch**	permissive (file check at adopt)	MCS/SPA multiple-testing (G4) + GARCH (E5)	**adopt, W1**
Research math	**scikit-learn · PyTorch · Stable-Baselines3**	BSD/BSD/MIT	E3 model training (offline nightly, Colab T4 lane)	locked (inside our E3 wrapper)
Research math	**NumPy / Numba**	BSD	swarm-harness vectorization (§8.7) — our engine, their kernels	locked
Research harness	**qlib** (Microsoft)	MIT	Mode R dataset→model→eval workflow — pin tagged release	candidate, W1 decision
Backtest honesty	**hftbacktest**	MIT	fill-model checklist for E4 (feed latency + order latency + queue position); direct code use = crypto venues only	reference, W1
Execution fallback	**NautilusTrader**	LGPL-3.0	documented E6 fallback **only if cTrader fails O8 verification**	fallback, not adopted

10.2 What WE build (own code — non-negotiable, no OSS substitute)

Piece	Why it can't be bought/borrowed
R1 data rail (ingest → validate → as-of/PIT stamp → rights tag)	PIT + rights discipline is the company's core IP (FULL-SYSTEM-PLAN R1/R4/R5)
E1 feature engine (Rust, in-proc, pre-warmed)	<1 ms budget; identical code must run in backtest AND live
E2 artifact evaluator (Rust + compiled formulas/ONNX)	frozen, signed, bit-exact runtime — no framework hooks in the hot path
E3 training orchestration (wraps the OSS libs above)	our pipeline, their math: data splits, gates, scheduling
E4 gate arena (CPCV + DSR ≥ 0.95 + crisis replay + swarm harness §8.7)	the promotion gate is the moat — an OSS tool must never grade its own homework
E5 governor (deterministic rules, zero ML)	risk policy: stops, size caps, news ladder, kill-switch, ban checks
E6 execution (scheduler · paper/shadow modes · venue adapters for Upstox/cTrader/Hyperliquid · idempotent order IDs)	venue-specific, safety-critical, zero-order-authority shadow mode is ours
E7 ledger + cost model + fiscal-year CA exporter	append-only ≥5 yr, native-currency, audit format
D5 artifact registry (sign + rollback) and **G4 trial log**	provenance and multiple-testing bookkeeping
PIT replay service + **Row 1 rights firewall** (Stage 7)	PIT correctness and license enforcement are ours alone

10.3 How they connect — the five interfaces

#	Interface	From (OSS) → To (ours)	Contract
1	**Storage contract**	venues → **our R1 rail** → OSS stores (D0–D6)	only the rail writes raw market data; engines read via pre-warm (D1/D2) or columnar scans (D6)
2	**Research contract**	PySR/MAPIE/arch/qlib/E3 run **inside Mode R** as libraries	they output candidates (formulas, models, intervals, test stats) — **never** artifacts; output goes to our E4 gate
3	**Artifact contract**	E4 passes → **our D5** signs (feature-hash + schema-version) → **our E2** compiles/loads at boot	a PySR formula becomes a frozen Rust computation; MAPIE interval ships with it; OSS is consumed only as *compiled output* at runtime
4	**Execution contract**	**our E6** → venue APIs (Upstox REST/WebSocket, cTrader OpenAPI, Hyperliquid API)	sole order path; NautilusTrader sits behind this interface as swap-in fallback (LGPL, unmodified)
5	**Audit contract**	everything (our engines **and** OSS-influenced runs) → **D4 append-only ledger + G4 trial log**	≥5 yr, every trial numbered, CA export

Dataflow in one line: `venue → R1 rail → OSS stores → (pre-warm) our E1/E2/E5/E6 → venue` for live; `OSS stores → our Mode R + OSS math libs → our E4 gate → our D5 → our E2` for learning. OSS sits **under** (storage) and **behind** (offline research) our code; the hot path and every promotion decision contain only our code.

---

§11 · Execution-engine integration verdict (Session 36 deep pull)

(Full evidence: `Research/notes/SESSION-36-ENGINE-INTEGRATION-RESEARCH.md` — GitHub API, READMEs, production field report, community issues.)

Engine	Built for bots?	Our agents on top?	Verdict
NautilusTrader (30.9k★, LGPL-3.0, Rust core)	Not a bot — the **engine under bots** (event-driven core: matching, book, clocks, message bus)	**YES, by design**: our cell = a `Strategy`/`Actor` class; full instruction range verified (IOC/FOK/GTC/GTD/DAY · post-only/reduce-only/icebergs · OCO/OUO/OTO)	**ADOPT** — E4-BT event-driven gear (honest FillModel/LatencyModel/queue matching) + execution kernel candidate under E6 where adapters exist (**Hyperliquid stable**)
Freqtrade (56k★, **GPL-3.0**)	**YES — packaged crypto bot** (1 strategy × pair whitelist, Telegram/webUI, sqlite ledger)	Technically yes (an agent can be a strategy) — but its shell/gates/ledger contradict the 75-cell fleet; candle-level fills, no queue/latency model (GH #10139/#7768)	**NOT ADOPTED** — isolated scout at most; crypto execution covered by Nautilus-Hyperliquid anyway
Hummingbot (20.7k★, Apache-2.0)	**YES — bot framework** for MM/arbitrage (V2 executors: arb, XEMM, grid, TWAP; Hyperliquid sponsor)	Drivable via API/CLI — but its **Condor = LLM decision loop → R2/R3 violation**	**PARKED** — the specialist executor IF crypto-MM is un-deferred; Condor = pattern reference only
MetaTrader5 Python	terminal+bot	**Windows-only package** — no native Linux	**REJECT** — extra OS + broker stack
cTrader Open API	vendor API	our E6 adapter (O2, A2/A3)	**KEEP — sole FX paper+live path**

The merge (owner's "combine as one system"):

Layer	Choice
Data rail + stores D0–D6 + domain engine E1/E2/E5/E7 + gates E4/G4 + D4/D5	**ours — not merged, this is the kept "domain engine"**
E4-BT event-driven gear	**Nautilus sim engine** (queue-position matching; hftbacktest checklist = acceptance test; swarm §8.7 unaffected)
E6 execution	**our interface on top** (idempotent IDs, E5, kill) · Nautilus rides **under** it for Hyperliquid (native adapter); **Upstox/cTrader = our adapters or Nautilus plugins → W2 spike decides** (measure P50/P99 + backtest-vs-shadow fill divergence; winner owns crypto execution)
FX research/backtest	Nautilus fed our own FX ticks (field-report precedent: GBP/USD)
Crypto MM/arb (deferred)	Hummingbot behind our API — parked
Freqtrade	out

Latency/quality basis (measured at W2, never claimed): Rust core + honest fills + backtest==live single codebase (18-month field report) — but Nautilus strategy callbacks are **Python** → benchmark against our pure-Rust E6 path in the W2 spike before choosing.

FINALIZED (owner, Session 37): Nautilus adoption is **decided** — (a) E4-BT event-driven engine = confirmed, (b) execution kernel under E6 wherever its native adapter exists (Hyperliquid). The only thing still open is *where it rides live* (W2 spike vs our own path) — not *whether* we use it. FX/stocks execution unaffected (our adapters).

---

§12 · Master engine checklist — adopted · to-build · open (Session 37)

#	Engine / store	Status	Next step (wave)
A · Adopted open-source (license-checked)			
A1	**NautilusTrader** — E4-BT event-driven gear + HL execution kernel	**FINALIZED (S37)**	integrate W1–W2 · W2 spike for crypto-live ownership
A2	PySR · MAPIE · arch — Mode R search/interval/gate math	adopted (S31)	W1
A3	sklearn · PyTorch · SB3 · NumPy/Numba — E3 math + swarm kernels	adopted	W1 (E3 lane = Colab T4)
A4	DuckDB (D6) · PostgreSQL (D3/D4) · Parquet (D0)	locked	W0a–W1
A5	hftbacktest — fill-model acceptance checklist	adopted (S31)	W1 (applied to Nautilus sim)
A6	qlib — Mode R harness candidate (pin tagged release)	open-adopt	decide W1
A7	NATS JetStream (D2)	default	confirm W0b micro-bench (G2)
A8	**Vectorized research gear: VectorBT (open, Apache 2.0) vs qlib** — Mode R sweep engine	open-adopt (S39, **C9**)	W1 bench
A9	QuantLib (options pricing math) · statsmodels · SALib · QuantStats · **xgboost + lightgbm**	adopt (S39, §14)	W1
A10	exchange_calendars (NSE ✓) · pandera (R1 validation cand) · ccxt + Spotware OpenApiPy/cTraderFixPy + Upstox SDK (adapter refs)	adopt (S39, §14)	W0a–W2
A11	Ops: Prometheus + OpenTelemetry (**Grafana AGPL = internal only**, never on Row 1)	adopt (S39, §14)	ops stage
B · Build ourselves (the domain engine — no OSS substitute)			
B1	R1 data rail (ingest→validate→as-of/PIT→rights) + PIT replay	to build	**W0a**
B2	D1 time-series store pick (QuestDB/Timescale/ClickHouse)	to measure	**W0a benchmark (G1)**
B3	E1 features (Rust, pre-warmed) · E2 evaluator · E5 governor · E7 ledger+cost+CA exporter	to build	**W0b**
B4	D5 signed artifact registry + D4 append-only + G4 trial log	to build	**W0b**
B5	E4-SIM swarm harness (10^4 worlds) · E4-BT vectorized gear · E4-WF (CPCV/DSR)	to build	**W1**
B6	E6-PAPER · central scheduler · L0–L4 cascade	to build	**W1**
B7	E6-SHADOW → E6-LIVE + **venue adapters — built by us AS Nautilus adapter plugins** (Upstox/Indian · cTrader)	to build	**W2–W3** (S38)
B8	E1 microstructure block (orderflow/footprint/VWAP = the "volume engine")	to build	**W4** (needs tick+depth feed)
B9	Row 1 serving replica + rights firewall	to build	**Stage 7**
C · Open / conditional / parked (real remaining unknowns)			
C1	Crypto execution owner: our Rust E6 vs Nautilus-on-HL	**DECIDED (S38): Nautilus** — estimate: delta ≤0.2% of venue RTT; W2 spike = validation	W2 validate

#	Engine / store	Status	Next step (wave)
C2	Upstox/Indian-broker route	**DECIDED (S38): build our own Nautilus adapters**	W2–W3 build
C3	cTrader (+MT5) route	**DECIDED (S38): build Nautilus cTrader adapter**; MT5 only if a broker forces it (Windows terminal; community adapter = reference)	W2–W3 build
C4	Hummingbot (MM/arbitrage executor)	**DROPPED (S38, owner) — no hummingbot; everything through Nautilus**	closed
C5	Freqtrade	rejected (scout-at-most)	—
C6	MT5	rejected (Windows)	—
C7	News Engine · World Engine (Row 2's other two)	**data pipelines PULLED INTO the Row2+3 build (S41 §16)** — collectors run W0b–W1 in parallel; display engines (Monitor/AI features) still last	collect W0b–W1 · display final stages
C8	World Monitor · Cockpit · Media hub	last, per owner's build order	final stages
C9	**Vectorized research gear owner: VectorBT open vs qlib vs our numba prototype** (S39)	open — **W1 bench** on our data	W1

Reading of the list: A = done deciding · B = the build (W0a → Stage 7) · C = the only genuinely open engine decisions (C9 resolves at W1, C1–C3 at the W2 spike; C4–C8 are parked/deferred by design).

---

§13 · Rust vs Nautilus speed verdict + capacity/parallel/sandbox answers (Session 38)

(Full evidence + arithmetic: `Research/notes/SESSION-38-NAUTILUS-PERF-CAPACITY.md`. All numbers = ESTIMATES with stated assumptions; W2 spike converts them to measurements.)

13.1 The speed calculation (crypto, per decision)

Both paths are Rust at the core; only the decision layer differs (pure-Rust in-proc = **2–20 µs** vs Nautilus Python callback = **20–200 µs typical**, +ms GIL jitter under bursts). Against venue RTT:

Route	Venue RTT	Rust share	Nautilus share	Winner by speed
Hyperliquid public API	~600–700 ms	0.001%	0.015%	tie — noise
Upstox	<45 ms claim	0.02%	0.2%	tie — noise
cTrader (AWS GA)	~10–50 ms (measure)	<0.1%	~0.5%	tie — noise
HL in-house node (W5, unverified)	maybe 1–5 ms	0.2–1%	5–50%	**Rust (colo only)**

VERDICT (owner condition resolved): NAUTILUS WINS for our scope — at every venue we will actually trade (W1–W6), the path difference is ≤0.2% of one round trip, while Nautilus buys honest fills, backtest==live parity, and a native Hyperliquid adapter. Rust speed matters only for colocated crypto MM (deferred job) → pure-Rust E6 stays as fallback + future contingency, not primary. **Decisions triggered (owner's "if nautilus wins"):**

1. **Build our own Nautilus adapters:** Upstox + other Indian brokers · **cTrader** · **MT5 only if a broker forces it** (Windows terminal caveat; community `nautilus_mt5/NT_MT5_Adapter` as reference).
2. **Hummingbot: DROPPED entirely** — all execution through Nautilus (C4 closed).
3. W2 spike remains — as **validation** of these estimates (P50/P99 + burst behavior), not as an ownership contest.

13.2 Capacity — how many parallel agents can one Nautilus engine hold?

Question	Answer (evidence)
Hard max cap?	**No published cap**; official scope = single-node small teams. Real bound = **Python-callback budget ≈20–50k callbacks/s per node process** (GIL, ESTIMATE) + venue API rate limits shared on one connection
Our 75 cells fit?	**Yes:** bar-driven (1 msg/s/cell ≈ 75 callbacks/s) → one node barely busy. Raw-tick-driven at crypto peak ≈ 50–75k deliveries/s → **shard 3–6 node processes**
Official support	"Add multiple strategies to a single node, or run additional nodes in **separate processes**" (live-node docs) · **one node per process** (singleton) · community precedent = single digits to ~33 live strategies
Numbers to prove it	W2/W3: callback budget under load + venue rate-limit headroom (measured, not claimed)

13.3 Does parallel drop latency? — NO

Parallel gives **throughput, backtest wall-clock and crash isolation — never lower per-order latency** (that = fixed processing + venue RTT; the GIL serializes callbacks inside a process, so adding cells to one node can only *raise* jitter under bursts). Latency falls only via: less processing (Rust) · less distance (colo) · shorter venue RTT (node).

13.4 One broker adapter for all agents? — YES, that is the Nautilus design

One exec client per venue; every strategy's orders carry its `strategy_id`; the venue sees **one connection**; portfolio/cache aggregates; several agents MAY share the same symbol (instances supported). Watch-outs: shared venue rate limits · one bad strategy disturbs co-located ones → sharding · restart drill per strategy before capital (field-report practice) · our E5 kill = flatten-all per venue + supervisor above the node.

13.5 "Multiple users in the same engine, own sandbox"? — soft yes in-process; real sandbox = separate processes

Model	Isolation
strategies share one node/process	soft (shared bus/cache/portfolio — blast radius = whole node)
node-per-process shards (recommended topology)	**hard** — matches official guidance AND blueprint G6/process-isolation + supervisor + monthly kill-drill
backtest sandbox	SIM venues + seed-replay (native) · live sandbox = paper/demo node with own credentials (E6-PAPER/SHADOW)
per-agent OS sandbox	our ops layer: Docker per shard

Recorded topology: minimum **3 venue nodes (Upstox · cTrader · Hyperliquid)**, each its own process/connection/kill; crypto shards by ~15–20 active cells; supervisor restarts.

---

§14 · Full engine/tool candidate pool beyond §12 (Session 39)

*(Owner: "what else do we need like vectorbt and other engines — list me all." Full evidence + licenses: `Research/notes/SESSION-39-VECTORBT-AND-ENGINE-POOL.md`. S35 rule holds: the pool below = research math, storage, reference SDKs, ops tooling — **no OSS replaces our gates or hot path**.)*

14.1 VectorBT verdict (the named one)

Verified from the repo itself: **Apache 2.0** · 9.6k stars · alive (1,130 commits) · NumPy + **Numba + optional Rust engine** · broadcasting **parameter sweeps** (thousands of variants × assets in one pipeline) · QuantStats integration · PRO = **paid** membership (parallelization +100 features; expiry keeps the installed copy alive).

VERDICT: adopt-candidate (open edition) for the E4-BT *vectorized research gear* — bench at W1 vs qlib vs our numba prototype (new item C9). Production validation gear + backtest==live parity gate stays ours (S35 moat rule). PRO is not the default — only if the open edition measurably fails the W1 bench (owner-priced option). Watch: open edition is single-process (fine — our sharding) · pandas memory bounds at tick-depth → feed from Parquet/DuckDB columns, never full loads.

14.2 The rest of the pool (license-verified where risky)

Layer	Adopt	Reject / defer (why)
Backtest — vectorized	**VectorBT open** (C9) · qlib (A6) · own numba gear (B5)	—
Backtest — classic	—	**backtrader = GPL-3.0 + unmaintained** · zipline-reloaded = US-equity only · backtesting.py/Jesse = too simple/packaged bot · LEAN = heavy C# platform · **mlfinlab = paid** (public repo stale) → CPCV stays ours
Options pricing	**QuantLib** (modified BSD, active) — Greeks/vol math behind our thin E1 wrapper	own pricing math from scratch (black-hole risk)
HPO / sweeps	**Optuna** (MIT) — Mode R sweeps + E3 HPO	Ray Tune/Dask (overkill — process sharding suffices) · hyperopt/nevergrad (Optuna covers)
Stats	**statsmodels** (cointegration/VAR/ADF) · **SALib** (Sobol robustness for E2) · optional bootstrapped/pingouin	—
Tearsheets	**QuantStats** (Apache) — research only	production cost/CA export stays E7-ours
ML	**xgboost + lightgbm** (tabular-alpha workhorses — were missing) · sklearn/PyTorch/SB3/Numba (A3)	river/tsfresh/JAX = defer until a need appears
Crypto data	**ccxt** (MIT, 104+ exchanges, incl. Rust) — Row 2 multi-exchange data + adapter reference; HL native (Nautilus) stays primary. Bonus: ccxt lists prediction markets → possible Kalshi rate-limit workaround (verify W1)	—
India data	Upstox official SDK (adapter reference) · jugaad-data/akshare (Row 3 historical backfill only; NSE ToS risk = P1/P8)	any of it on the hot path
Calendars	**exchange_calendars** (Apache, 50+ exchanges, NSE ✓) — R1 session engine	—
Data quality (R1)	**pandera** (MIT, pandas/Polars) as validation layer — bench vs hand-rolled Rust at W0a	Great Expectations (heavy + stewardship churn FICO→Fivetran 2026) · evidently (defer with E7)
cTrader build	**Spotware OpenApiPy + cTraderFixPy** (MIT, official — REST/WS + FIX) as build blocks for the B7 adapter	—
Ops metrics	Prometheus + OpenTelemetry (Apache)	**Grafana = AGPL → internal ops only, never exposed on Row 1**
Orchestration	— (R1 rail = simple pipelines; live scheduling = our B6)	Prefect/Dagster/Temporal/Airflow = defer
Row 1 reference	OpenBB (license verify if ever used)	as a base — Row 1 = our own build (Stage 7)

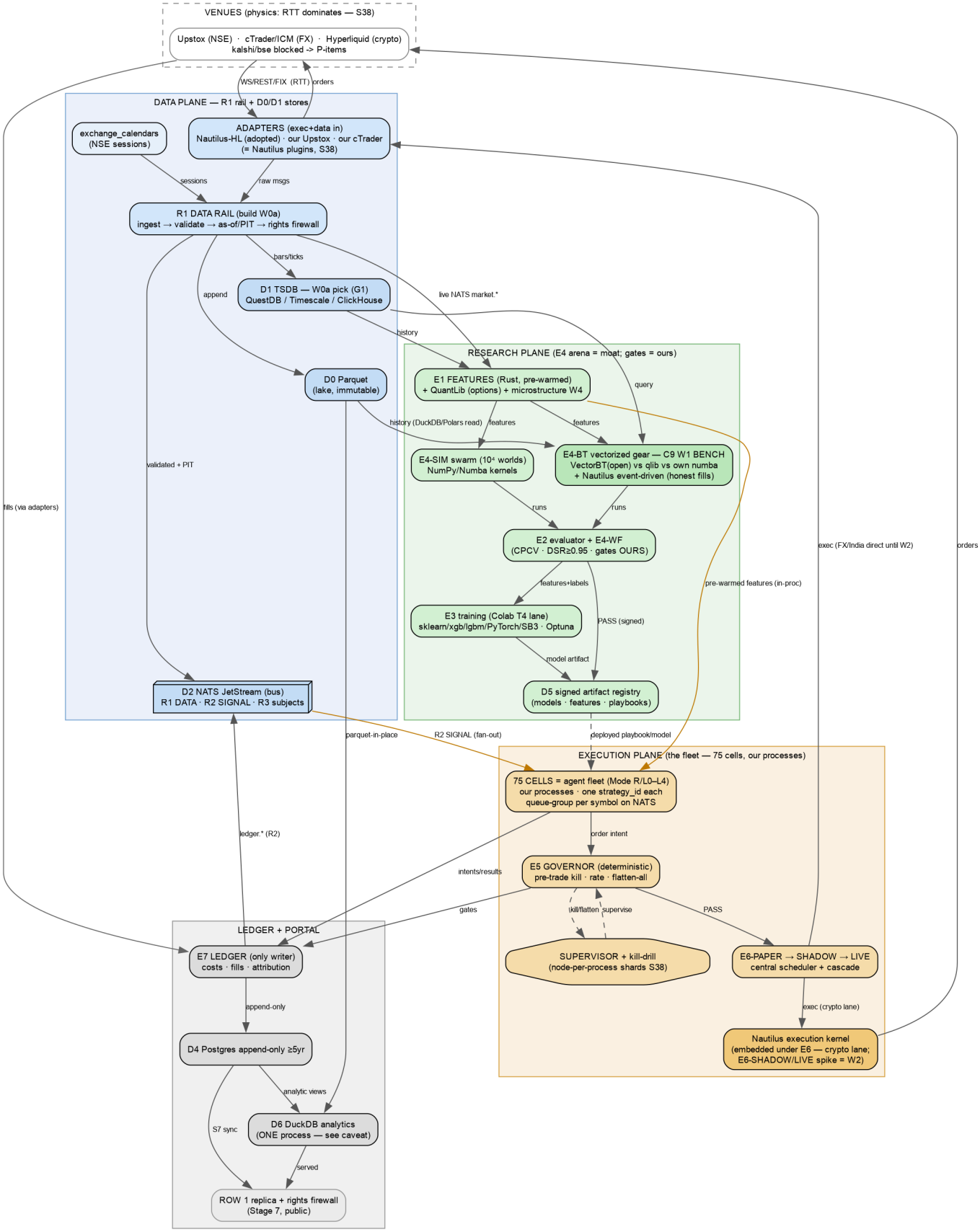
Net: 11 new adopts/candidates recorded (A8–A11) · one new open item (**C9**, W1 bench) · nothing blocks **W0a**.

---

§15 · Complete wire structure + agent-capability check (Session 40)

(Owner: "how do all these engines/stores/our builds wire and communicate — and can they run our agents, everything is built for bots?" Evidence: `Research/notes/SESSION-40-WIRING-AND-AGENT-CAPABILITY.md`.)

15.1 The whole wire in one picture



15.2 The five transports (the only ways anything talks)

#	Transport	Carries	Pattern
1	**D2 NATS bus** (core fan-out + JetStream durable)	R1 `market.*` · R2 `signal./order./risk./ledger.*` · R3	queue groups (one delivery per group → load-balance cells per symbol) · durable consumers (independent replay) · partitioned (per-key ordering)
2	Files — **D0 Parquet / D5 artifacts**	history lake, signed models/playbooks	single writer (R1/E7), unlimited readers
3	SQL — **D1 TSDB · D3/D4 Postgres**	feature/history, reference, append-only ledger	multi-client servers; concurrent headroom = **G1 bench criterion (W0a)**
4	Embedded — **D6 DuckDB**	analytics/tearsheets	**single-process only** (official docs) — one D6 process, reads Parquet in place
5	In-proc calls	E1 features · E4 engines (vectorbt/qlib/numba) · QuantLib · gates	plain function/FFI calls, µs-class — no store query per tick (S34 rule)

15.3 Live hot path (one order)

venue → **adapter** (Nautilus-HL or ours) → **R1 rail** (validate/PIT/rights) → NATS `market.*` → **cell** (pre-warmed features, 2–20µs decision) → **E5 governor** (µs, deterministic) → **E6** (Nautilus kernel crypto lane / direct FX+India until W2) → venue *(RTT dominates — S38)* · async: fill → **E7** → D4 + `ledger.*`.

15.4 Research path (the C9 engines are LIBRARY CALLS, not bots)

D0/D1 → **E4-BT sweeps** (vectorbt/qlib/numba — C9 picks at W1, called as functions in OUR worker processes; qlib's `qrun` is an optional wrapper) → survivors re-run on **Nautilus event-driven** honest fills (hftbacktest checklist) → **E2/E4-WF gates** (CPCV · DSR ≥0.95 — ours) → **D5 signed** → **E6-PAPER**.

15.5 Agent-capability check — "everything is built for bots?"

Component	Built for	Runs our 75-agent fleet?
Nautilus	single-bot `TradingNode` by default	**YES as engine under E6** — official README: "user-defined components, assemble entire systems from scratch using cache + message bus" · multi-strategy node · node-per-process shards (S38)
vectorbt · qlib · numba	research libs	**YES — import-and-call** inside our own E4 workers (no harness)
NATS	microservices	**YES natively — it IS the agent backbone** (queue groups/durable consumers)
TSDB · Postgres · Parquet	servers/files	YES — concurrent clients (headroom measured at G1)
DuckDB	embedded analytics	**NOT a shared server** → sits behind **ONE D6 process**; reads Parquet in place (official: single-process concurrency) — never in any critical path
Our E/R engines	—	designed for it (S35 five interfaces: storage · research · artifact · execution · audit)

The inversion: our agents are NOT run inside bot frameworks — the frameworks come **inside our fleet as libraries + execution kernels**, glued by NATS, state-fed by D0/D1, audited by D4/E7. Failure domains: one Nautilus node per venue shard (+crypto shards ~15–20 cells), non-crypto cells never touch Nautilus, supervisor kill-drill above all (d8 diagram).

---

§16 · Session 41 — Row 2+3 ONE build · three-domain warehouse · PIT time machine · LLM scoring (scope decision)

16.1 What was decided (owner, 2026-10-28)

Element	Decision
Build scope	**Row 2 + Row 3 financial trading = ONE integrated build** (one repo · one rail · one schema). Row 1 + World Monitor / AI-assistant *features* = **HELD** (built last)
Row 2 contents	ALL data engines in Row 2: **financial** = entire world markets, high-quality point-level, organized · **world-monitor** = pull all, organize · **news** = pull all, organize — one warehouse, one cleaner, domain-partitioned (`market/ news/ world/`)
Row 3 consumption	**LLM agents = enrichment**: read raw news/fundamentals/world data → organize → **make scores** → scores written back into Row 2. **Non-LLM agents = traders**: read prices + precomputed scores (LLM never inline in the hot path)
One data, four uses	**train · backtest · research · trade on the SAME warehouse** — no train/live skew (the classic backtest≠live killer)
PIT time machine	pick any instrument, any past timestamp → live-level context known *at that moment*: ticks/candles/volume/technicals + news published before it + fundamentals as-of + scores — **no future leakage**. PIT rule extends from prices to **ALL domains**
Assets folded in	`alphalens-*` repos (deeper gathering already planned there) + `Research/` PDF books corpus
Serving	after build: **clone** the warehouse (file copies + read replica + D6) → Row 1 Terminal + held features; original stays **single-writer**
Latency logic	co-build removes Row2↔Row3 integration hops — direction true; exact numbers **measured at W0a/W2, never claimed**

16.2 Feasibility check ("good idea or not?" — numbers = ESTIMATE unless marked)

Finding	Number / verdict
Architecture	✅ right pattern — central PIT warehouse + enrichment layer + serving clone is how institutional shops structure this; single-writer/clone discipline already in §15
"Entire world **pin-to-pin**" as blanket scope	❌ **infeasible as stated**: US tape alone ≈10–12B quote/trade events/day → ≈1–3.5 TB/day → **≈400–1000+ TB/year** (petabyte budget)
Our **traded** set, full tick+depth (59 NSE · 13 FX · 3 crypto)	**≈0.1–0.3 TB/year** compressed (Parquet+zstd) — trivial
World structured (≈100k instruments: daily bars; where free → 1-min; reference; econ releases)	**≈50–300 GB/year** — trivial; powers "pick any stock any date"
News	metadata/headlines/summaries ≈10–50 GB/year; full text = per-source terms; **Row 1 shows headline+summary+link** (rights firewall)
World data	mostly public datasets, GBs/year, free with attribution (verify per source)
PIT fundamentals	⚠️ hard — as-reported *vintage* data scarce/expensive → new **O12**
LLM scoring cost	score only trading-relevant universe + macro events (10^2–10^3 items/day) ≈ cents–dollars/day on small models; **scores versioned in D5**; **E2 must show score→edge lift** before scores enter any gate
Schedule	⚠️ main risk = news/world collection blocking trading waves → staged below

16.3 Verdict + the four amendments (applied)

VERDICT: GO — good idea with 4 amendments:

#	Amendment
A · Tier the world	**T1** full tick+depth = our tradable set (0.1–0.3 TB/yr) · **T2** world structured daily/1-min + reference (50–300 GB/yr) · **T3** world pin-to-pin = **on-demand per research need, never blanket-collected**
B · Stage it	W0a = market rail first (unchanged) · **world + news collectors = W0b–W1 in parallel** (never block paper trading) · scoring layer = W1–W2 · display engines held (last)
C · Honest gaps	PIT vintages + news/world rights = open items **O12/O13**; gaps marked in-DB, never silently filled
D · Scores must earn	scores = async enrichment (out of hot path) + **E2 edge-lift gate** + D5 versioning; decorative scoring rejected by the gate

Row changed: C7 — news/world **data** pulled into this build (W0b–W1); display engines still last.

§17 · Session 42 — licensing stance + backfill-first build order

17.1 Owner instructions (2026-10-28)

#	Instruction
1	"We don't sell anything with this system, so we don't bother about copyrights, licensing, etc."
2	"First **build this and pull 6–10 years of data for the entire system** (world financial market data · news · world monitor) — **then** build and train agents — **then** increase historical data slowly."

17.2 Licensing — copyright review DROPPED, three practical carve-outs kept

System is private, nothing sold, Row 1 held → legal/licensing review downgraded from a gate to an operational checklist. **But three risks that are NOT copyright still bite a private system:**

#	Risk	What it actually is	Already hit?	Handling
1	**Venue API ToS**	terms of service — breaking them = instant **account/IP ban**, not a lawsuit; applies to *pulling data*	✅ yes — Kalshi rate-limited, BSE 403, CME 403 during planning alone	**O13 reframed = ToS/ban-avoidance checklist**: official endpoints · stated rate limits · honest UA · paid tier when free refuses (data cost = budget item, not legal risk)
2	**Row 1 public display**	exchanges license *display* of their quotes — applies even to **free, non-sold** products	not yet (Row 1 held)	Stage 7 only: delayed-vs-realtime entitlement + rights firewall already designed; a checklist row, never a build blocker now
3	**Regulation — FIU-IND / FEMA**	trading regulation — applies because we **trade**, not because we sell	open — **O11**	unchanged: hard pre-live gate for Hyperliquid/crypto from India

Dropped: copyright review of *storing + computing on* private copies (no distribution, no sale, no public display until Stage 7).

17.3 Backfill-first build order (amends §16.3 amendment B)

Owner order = **data first → agents second → history grows third.** Applied as an explicit gate:

Step	Wave	What happens	Gate
1	**W0a** (unchanged)	build stores + R1 rail + one venue live path — a place to put the history	G1 benches
2	**B1 · BACKFILL (new)**	pull **6–10 years of ALL three domains**, tiered T1/T2/T3 (§16.3-A), into `market/ news/ world/`	**B-GATE: domain backfill verified + gaps marked before any agent is TRAINED on it**
3	**W0b–W1**	build agents and **train** on the backfilled warehouse. *(Agent scaffolding/code may be written during B1 — it needs no data; TRAINING waits.)*	existing E2/E4 gates (DSR ≥0.95)
4	**W4+**	"increase history slowly" = mostly free: the live rail keeps appending every session	PIT/replay checks

Supersedes the collector half of Amendment B (world+news collectors at W0b–W1 *parallel to agents*): collectors now run inside **B1, before agent training**. Tiers (A), honest-gaps (C), scores-must-earn (D) unchanged. New table row **B1** added to §8.

17.4 What "6–10 years" actually exists — availability table (verified this session where marked)

Domain	Slice	History actually available	Evidence
Market · NSE	**daily bars**	✅ **2000+ (≈26y)** — Upstox V3, decade chunks	Upstox V3 docs **VERIFIED (S14)**
Market · NSE	**1-min bars**	⚠️ **only Jan 2022+ (≈4y)** on V3; V2 = last 6 months — **pre-2022 gap**	Upstox V3/V2 docs + community **VERIFIED (S14)**
Market · NSE	weekly / monthly	✅ 10y+ (no retrieval limit)	Upstox docs **VERIFIED (S14)**
Market · NSE	sub-minute pre-2022	❌ not free → **paid historical vendors** (unverified candidates: Global-Datafeeds, TrueData) → decide at B1 sourcing	gap → B1 vendor pick
Market · FX	tick/bar archive	✅ 10y+ free archive sources exist (Dukascopy/HistData class) — exact depth measured at pull	partially verified — **measure at B1**
Market · Crypto	OHLCV from venue listing	✅ deep for majors (2017+ free archives, older on some venues); ⚠️ **Hyperliquid perps = 2023+ only**, **HYPE ≈2024–25 only** — inherently short	venue history (S38)
News	headlines / metadata	✅ **GDELT 2015+ (≈11y)** free; full text = per-source terms → store headline+summary+link	GDELT coverage **VERIFIED (S14)**
World monitor	macro/geopolitics, econ releases	✅ decades — public sources (World Bank/IMF/calendars)	public datasets
Fundamentals	as-reported PIT vintages	⚠️ scarce/expensive — **O12 open**; gaps marked in-DB	S41

Verdict: 6–10y is **real for daily bars · FX · news · world**; **partially real** for NSE 1-min (2022+, ~4y) and per-instrument crypto; the **one real hole = NSE sub-minute before 2022** → paid-vendor decision is the first B1 sourcing task. Gaps get marked in-DB, never silently filled (§16.3-C).

---

§18 · Session 43 — merged gathering TODO + storage target

18.1 Owner decisions (2026-10-28)

#	Decision	Applied
1	Merge all verified external resources into the existing dataset TODO, with links	18 rows added in `Research/data/todo/15-external-resources.md`; links are in each source cell and its URL footnotes. No separate resource list.
2	Accept the **16 TB** storage target	Use 16 TB as the 10-year planning ceiling for the lean 4–16 TB range, including 2× replication + backup. Re-measure at W0a/B1; this is a target, not a measured requirement.
3	Put the six HuggingFace datasets into **W0 backfill first**	Six dataset rows are W0 in sheet 15; the India news corpus is also W0 in sheet 14. Keep license-sensitive options data segregated.
4	Read alphalens deeply and produce one consolidated final TODO	Alphalens' India source catalog and market materials informed 89 gap-closure rows in sheet 14; prediction markets get a new 7-row sheet 13. All three sheets feed the existing generated master.

18.2 Gathering plan and merge outcome

The existing `Research/data/todo/DATASET-TODO.md` remains the single consolidated gathering plan. `Research/scripts/build_todo_merge.py` discovers numbered domain volumes, validates their 13-column rows, applies first-occurrence slug deduplication, and regenerates the Markdown master plus `Research/data/extras/data-catalog.jsonl`. New domain titles 13–15 are registered in that script; the generated files are not hand-edited.

Volume	Rows	Purpose
`13-prediction-markets.md`	7	Polymarket Gamma/CLOB, Kalshi, Manifold, Metaculus, PredictIt archive, India roll-up; resolves the previously empty prediction-market trunk (O1).
`14-india-depth.md`	89	Alphalens India gap-closure; rows cite relevant `in_*` source-catalog IDs.
`15-external-resources.md`	18	Six HuggingFace datasets plus 12 GitHub resources, with upstream links and row-level evidence/license notes.

Alphalens review covered its 166-surface India catalog, source-catalog evidence states, market files, volumes 02/10/11, global coverage, and spikes S1–S8. The catalog surfaces were cross-checked against the existing TODO; probe false positives were removed before adding India-depth rows. Evidence labels remain distinct: `VERIFIED-UPSTREAM-DOC`, `SURVEY-EVIDENCE`, and `UNVERIFIED-CANDIDATE` are not interchangeable.

18.3 Backfill and storage implications

- **W0/B1 intake:** start with the six external HF data sets identified in sheet 15: NSE 1-minute and daily bars (Jhanvi), independent India daily OHLCV (Vishnu), India index-options 1-minute bars (thetrademarkk), Hyperliquid perpetual history (Chainticks), and XAUUSD ticks (CarlosSilva1). The India news/ticker set (mveen3) is also W0 in sheet 14. Verify spans, schemas, completeness, PIT timestamps, and duplicate coverage on pull before training; a dataset-card license does not establish underlying source-data rights.
- **NSE minute-history gap:** Jhanvi is a candidate bridge for 2022 onward, not proof of pre-2022 availability. The pre-2022 paid-vendor decision remains a B1 sourcing task. Compare overlapping samples against the official exchange rail before treating a third-party copy as a trading spine.
- **Storage target:** use **16 TB** as the accepted upper planning target for the lean 10-year footprint including replication and backup (earlier lean range: 4–16 TB). Vol-11's post-backfill Vault estimate remains approximately 1.4–1.6 TB before broader market/news/world expansion. These are estimates; measure compressed bytes, replication, retention, and backup at W0a/B1.
- **License isolation:** CC-BY-NC index-options data must remain segregated from commercial/display-capable outputs. AGPL repositories are internal-use code references; NOASSERTION repositories are design-study only. Session 42's practical carve-outs (venue ToS, Row 1 display rights, FIU-IND/FEMA) remain unchanged.

Build evidence (2026-10-28): the merged master contains **1,114 rows**, with **0 schema problems** and **1 pre-existing duplicate** (FINRA short-sale in sheets 03 and 05; the first occurrence is retained). The TODO PDF was regenerated. See Session 43 in the planning log for resource-level verification and scan results. **Source: S15.**

Sources

ID	Source	Evidence class
S01	`FULL-SYSTEM-PLAN.md` §2 (three rows) · §6 (storage lanes: "evaluate QuestDB/TimescaleDB/ClickHouse before committing") · R8 (HFT honesty)	internal plan (owner-approved)
S02	`AGENT-FLEET-BLUEPRINT.md` §3 (E1–E7, R1–R3) · §8.7 (swarm harness) · §10 (latency budget) · §15 (business layer)	internal plan (council-validated)
S03	Session 18 planning log — original E1–E7 naming + old R1/R2/R3 rail draft	internal log
S04	Session 31 tool evaluation (`Research/notes/SESSION-31-TOOL-EVALUATION.md`) — hftbacktest fill-model checklist, PySR/MAPIE/arch adopt	web-verified (GitHub/API)
S05	Venue latency facts (Upstox <45 ms vendor claim · HL ~600–700 ms · cTrader GA)	web-verified vendor docs (V)
S06	Nautilus quality/docs pages + GH #3736/#3634/#1677 (benchmarks, multi-strategy node, one-node-per-process, single-threaded backtest, µs local latency)	AUTHORITATIVE-TECHNICAL / GitHub facts
S07	`SESSION-38-NAUTILUS-PERF-CAPACITY.md` — speed arithmetic, callback-budget estimate, community multi-strategy precedent	internal research (estimates labeled)
S08	Session 36 field report + community threads (restart drills, multi-strategy live experience)	community (EVIDENCE-INFERENCE)
S09	`SESSION-39-VECTORBT-AND-ENGINE-POOL.md` — repo-level license/maintenance verification (vectorbt, QuantLib, ccxt, QuantStats, Optuna, exchange_calendars, OpenApiPy, pandera vs GX, mlfinlab)	internal research (sources S01–S13 therein)
S10	2026 practitioner backtesting surveys (backtrader unmaintained+GPL, zipline-reloaded permissive-only classic)	web-verified secondary
S11	Nautilus README/Architecture/MessageBus ("user-defined components … assemble entire systems from scratch using the cache and message bus") · NATS queue-groups/JetStream-consumer docs · qlib `qrun`/library docs · DuckDB concurrency docs (single-process, single-writer)	AUTHORITATIVE-TECHNICAL
S12	`SESSION-40-WIRING-AND-AGENT-CAPABILITY.md` + diagram `diagrams/d8-engine-wiring.png` — complete wire, hot/research paths, agent-capability verdicts	internal research
S13	Session 41 owner scope decision + feasibility check (first-principles storage arithmetic; tiering/staging amendments A–D; O12/O13 added)	internal (ESTIMATES labeled)
S14	Upstox V3/V2 historical-candle docs (interval availability: minutes from **Jan 2022**, days from **Jan 2000**, weeks/months 10y+; retrieval chunk limits) · GDELT time coverage (2015+) · Session 42 owner instructions (licensing stance, backfill-first order)	AUTHORITATIVE-VENDOR (S14) + internal
S15	Session 43 owner decisions + alphalens 166-surface India catalog and market files · HuggingFace dataset cards · GitHub repository metadata; resource URLs and per-row evidence/license notes in `Research/data/todo/15-external-resources.md`	internal + upstream pages (URL/HTTP and build checks recorded in Session 43 log)